

chicago_tnp_price_analysis

June 29, 2026

1 Chicago TNP Trips — Price Dynamics Analysis

Split each fare into a rule-like **base** price and a demand-driven **variable** part, check that the variable part behaves like surge, and track how dynamic pricing changed from 2022 to 2024.

Data source: MatrixOne (`chicago_tnp.trips`), Chicago rideshare trips, 100k/month sample.

```
[1]: # Install dependencies (run once)
      %pip install -q pandas numpy scikit-learn matplotlib requests pymysql
```

[notice] A new release of pip is

available: 24.0 -> 26.1.2

[notice] To update, run:

```
pip3 install --upgrade pip
```

Note: you may need to restart the kernel to use updated packages.

1.1 1. Load trip-level data from MatrixOne

```
[2]: import pymysql
      import pandas as pd

      conn = pymysql.connect(host="127.0.0.1", port=6001,
                             user="root", password="111", database="chicago_tnp")

      # Pull trip-level rows. Drop pooled trips: their split fares distort per-trip
      # price.
      sql = """
      SELECT trip_start_timestamp, trip_miles, trip_seconds,
             fare, additional_charges, trip_total,
             pickup_h3, pickup_community_area
      FROM trips
      WHERE shared_trip_authorized = 0
      """

      df = pd.read_sql(sql, conn)
      conn.close()

      # DECIMAL columns come back as Python Decimal; cast to float so we can do math.
      for c in ["fare", "additional_charges", "trip_total"]:
```

```
df[c] = df[c].astype(float)

print("rows:", len(df))
df.head()
```

/var/folders/jt/cz2nbl8d6vs1vcffdydk4n5440000gn/T/ipykernel_54643/3335268121.py:15: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df = pd.read_sql(sql, conn)
```

rows: 3415423

```
[2]:  trip_start_timestamp  trip_miles  trip_seconds  fare  additional_charges  \
0   2022-03-10 08:30:00      17.9200          3043  62.5              5.00
1   2022-03-10 08:30:00       5.1046          1148  22.5              3.54
2   2022-03-10 08:30:00       5.5000          1061  20.0              1.02
3   2022-03-10 08:30:00       0.7000           201   7.5              1.02
4   2022-03-10 08:30:00       4.1400          1051  35.0              1.02

    trip_total  pickup_h3  pickup_community_area
0         67.50  89275934ed3ffff             76
1         26.04  892664ccd9bffff             46
2         21.02  892664c1607ffff              6
3          8.52  892664cb1a7ffff             19
4         36.02  892664caea3ffff             24
```

1.2 2. Time features

Turn the timestamp into simple, meaningful buckets (this is binning).

```
[3]: df["trip_start_timestamp"] = pd.to_datetime(df["trip_start_timestamp"])
dt = df["trip_start_timestamp"].dt

df["year"]      = dt.year
df["month"]     = dt.month
df["hour"]      = dt.hour
df["dow"]       = dt.dayofweek          # 0=Mon ... 6=Sun
df["is_weekend"] = (dt.dayofweek >= 5).astype("int8")
df["is_night"]  = ((dt.hour >= 22) | (dt.hour < 6)).astype("int8")
df["is_rush"]   = ((dt.dayofweek < 5) &
                  (((dt.hour >= 7) & (dt.hour < 10)) | ((dt.hour >= 16) & (dt.
→ hour < 19))))).astype("int8")
df[["year", "hour", "is_weekend", "is_night", "is_rush"]].head()
```

```
[3]:  year  hour  is_weekend  is_night  is_rush
0   2022     8           0           0         1
1   2022     8           0           0         1
```

2	2022	8	0	0	1
3	2022	8	0	0	1
4	2022	8	0	0	1

1.3 3. Split fare into base + variable

Idea: a fare should grow with distance and time. That predictable part is the **base** (a rule-like price). Whatever is left over is the **variable** part (demand-driven surge).

We fit the base **separately for each year**, so inflation is absorbed into each year's base, and the variable part is each trip's deviation from *that year's* rule price.

```
[4]: import numpy as np
from sklearn.linear_model import LinearRegression

df["base"] = np.nan
coefs = []
for y in sorted(df["year"].unique()):
    m = df["year"] == y
    model = LinearRegression().fit(df.loc[m, ["trip_miles", "trip_seconds"]],
    ↪df.loc[m, "fare"])
    df.loc[m, "base"] = model.predict(df.loc[m, ["trip_miles", "trip_seconds"]])
    coefs.append({"year": y,
                  "start_$": round(model.intercept_, 2),
                  "per_mile_$": round(model.coef_[0], 2),
                  "per_min_$": round(model.coef_[1] * 60, 2)}) # coef is per_
    ↪second -> *60

df["variable"] = df["fare"] - df["base"] # leftover = demand-driven part

# The coefficients look like a real fare table, and stay stable across years.
print(pd.DataFrame(coefs).to_string(index=False))
```

year	start_\$	per_mile_\$	per_min_\$
2022	6.27	0.91	0.38
2023	4.81	0.92	0.34
2024	4.00	1.05	0.34

1.4 4. Check that “variable” really is demand-driven

If it is surge, it should be **positive** when demand is high (rush, night) and **negative** when demand is low (midday).

```
[5]: print("mean variable by rush hour:")
print(df.groupby("is_rush")["variable"].mean().round(2))
print("\nmean variable by night:")
print(df.groupby("is_night")["variable"].mean().round(2))
```

mean variable by rush hour:

is_rush

0 -0.13

1 0.52

Name: variable, dtype: float64

mean variable by night:

is_night

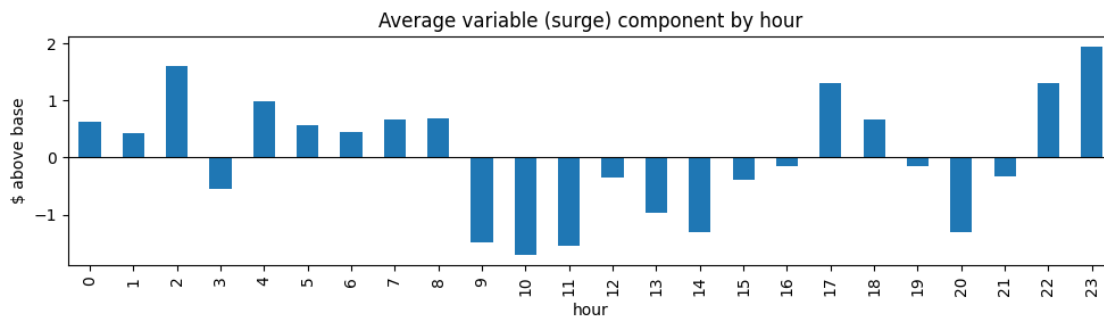
0 -0.34

1 0.95

Name: variable, dtype: float64

```
[6]: import matplotlib.pyplot as plt

df.groupby("hour")["variable"].mean().plot(kind="bar", figsize=(10, 3),
      title="Average variable (surge) component by hour")
plt.axhline(0, color="k", lw=0.8)
plt.ylabel("$ above base"); plt.xlabel("hour"); plt.tight_layout(); plt.show()
```



1.5 5. Compare prices across years

Compare years fairly by holding distance roughly fixed (same distance bucket). Each year has all 12 months sampled equally, so full-year vs full-year is season-balanced.

View A: nominal median fare per distance bucket, per year.

```
[7]: df["mile_bin"] = pd.cut(df["trip_miles"], bins=[0, 2, 5, 10, 20, 100])
df.pivot_table(values="fare", index="mile_bin", columns="year",
      aggfunc="median", observed=True).round(2)
```

```
[7]: year      2022   2023   2024
mile_bin
(0, 2]      7.5    7.5    7.5
(2, 5]     12.5   10.0   10.0
(5, 10]     17.5   15.0   15.0
(10, 20]    27.5   27.5   27.5
```

```
(20, 100] 40.0 37.5 40.0
```

View B: did the demand-driven premium change? Average nighttime variable per distance bucket, per year. A consistent drop across all buckets means the change is real, not driven by one trip type.

```
[9]: night = df[df["is_night"] == 1]
night.pivot_table(values="variable", index="mile_bin", columns="year",
                    aggfunc="mean", observed=True).round(2)
```

```
[8]: year      2022  2023  2024
mile_bin
(0, 2]      1.95  0.74  0.66
(2, 5]      2.78  1.15  0.43
(5, 10]     2.62  0.42 -0.89
(10, 20]    2.87  0.54 -0.14
(20, 100]   0.62 -0.75 -3.77
```

1.6 6. Yearly strength of dynamic pricing

Two simple per-year measures: - **night_premium**: average extra (vs base) that night trips cost
- **volatility_std**: spread of the variable part (bigger = more price swing)

```
[9]: strength = pd.DataFrame({
    "night_premium": df[df["is_night"] == 1].groupby("year")["variable"].mean(),
    "volatility_std": df.groupby("year")["variable"].std(),
}).round(2)
print(strength)
```

```
      night_premium  volatility_std
year
2022              2.48             8.39
2023              0.77             6.57
2024              0.05             6.70
```

1.7 7. Demand growth vs pricing strength

The 100k/month sample erased true volume, so pull real yearly totals from the Chicago data portal API and put them next to the pricing strength. Requires internet.

```
[10]: import requests

def yearly_population(dataset_id):
    url = f"https://data.cityofchicago.org/resource/{dataset_id}.json"
    params = {"$select": "date_extract_y(trip_start_timestamp) AS yr, count(*) AS n",
              "$group": "yr", "$order": "yr", "$limit": 5000}
    out = pd.DataFrame(requests.get(url, params=params).json())
    return out.astype({"yr": int, "n": int})
```

```

pop = pd.concat([yearly_population("m6dm-c72p"), # TNP trips 2018-2022
                 yearly_population("n26f-ihde")], # TNP trips 2023-2024
                 ignore_index=True)
pop = pop[pop["yr"].between(2022, 2024)].set_index("yr")
pop.index.name = "year"

# Demand growth (relative to 2022) next to the pricing-strength measures
compare = pop.join(strength)
compare["demand_vs_2022"] = (compare["n"] / compare.loc[2022, "n"]).round(2)
compare

```

```

[10]:
      n  night_premium  volatility_std  demand_vs_2022
year
2022  69109780         2.48           8.39           1.00
2023  83410973         0.77           6.57           1.21
2024  90958543         0.05           6.70           1.32

```

1.8 8. Summary

Finding. From 2022 to 2024, Chicago rideshare demand grew about 32% (real trips 69M -> 91M), but dynamic pricing got *weaker*, not stronger. The average night premium fell from about +\$2.48 per trip to +\$0.05 — consistently across every distance bucket — and overall price volatility narrowed (std 8.4 -> 6.7). In short: more demand, flatter pricing.

Method. For each year, fit $\text{fare} \sim \text{distance} + \text{time}$ (linear regression) to get a rule-like base price (about \$5 start + \$0.96/mile + \$0.35/min). $\text{variable} = \text{fare} - \text{base}$ is the leftover, demand-driven part; it was validated to move with demand (positive in rush hours and at night, negative midday). Real yearly volumes came from the city portal API, since the monthly 100k sampling removed absolute volume.

Limits. We can confirm *what* happened (premium fell, demand rose), but not *why* (supply recovery vs platform strategy) — that needs driver/supply data we don't have. Fares are rounded to \$2.50 and the linear base underestimates long trips, so premium magnitudes are approximate; the trend is reliable.